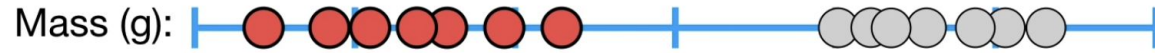


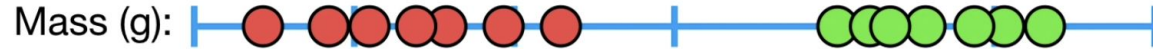
Support Vector Machine

Lecturer: Albert Yang (楊景明)

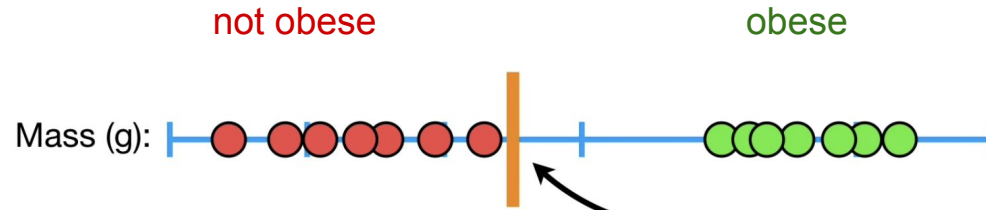
Example



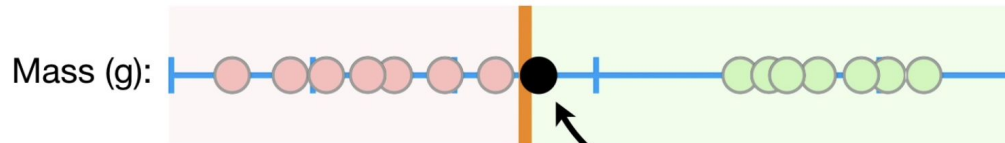
The **red dots** represent mice are **not obese**...



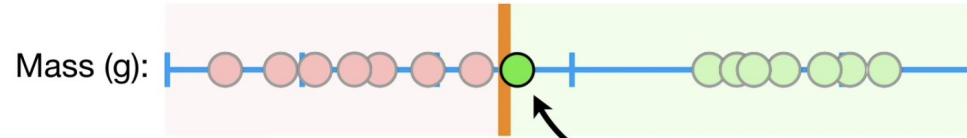
...and the **green dots** represent mice are **obese**.



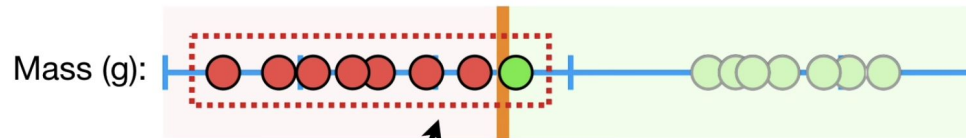
Based on these observations, we can pick a threshold...



However, what if get a new observation here?

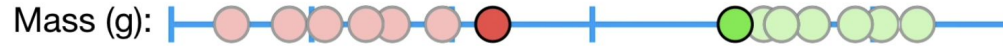


Because this observation has more mass than the threshold, we classify it as **obese**.

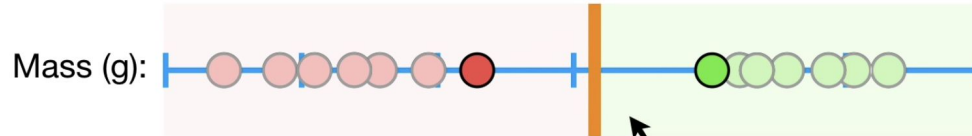


But that doesn't make sense, because it is much closer to the observations that are **not obese**.

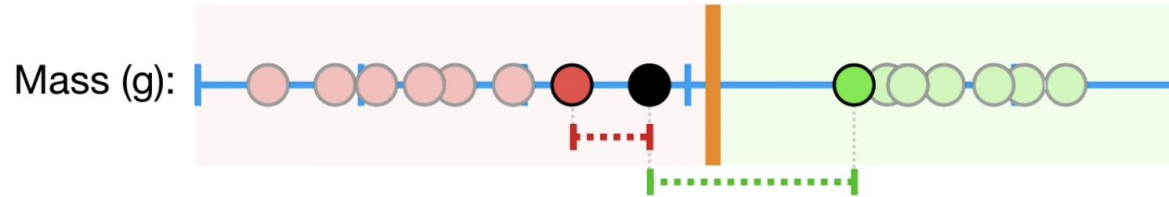
A Better Way



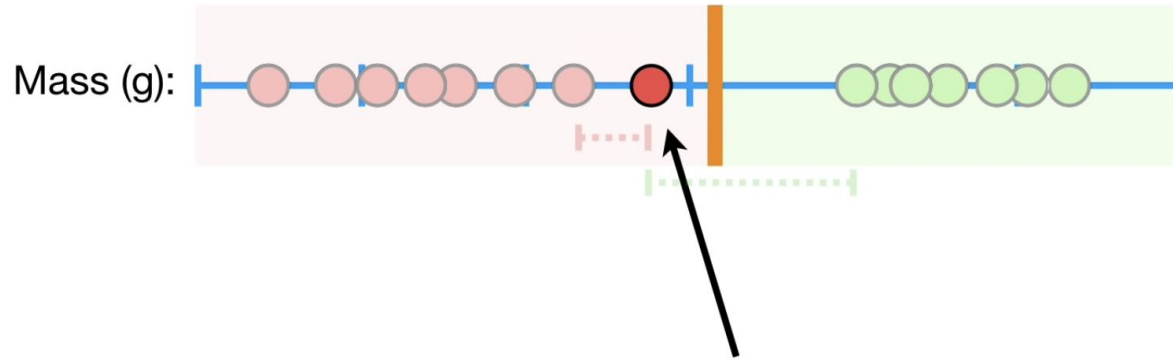
...we can focus on the observations on the edges of each cluster...



...and use the midpoint between them as the threshold.

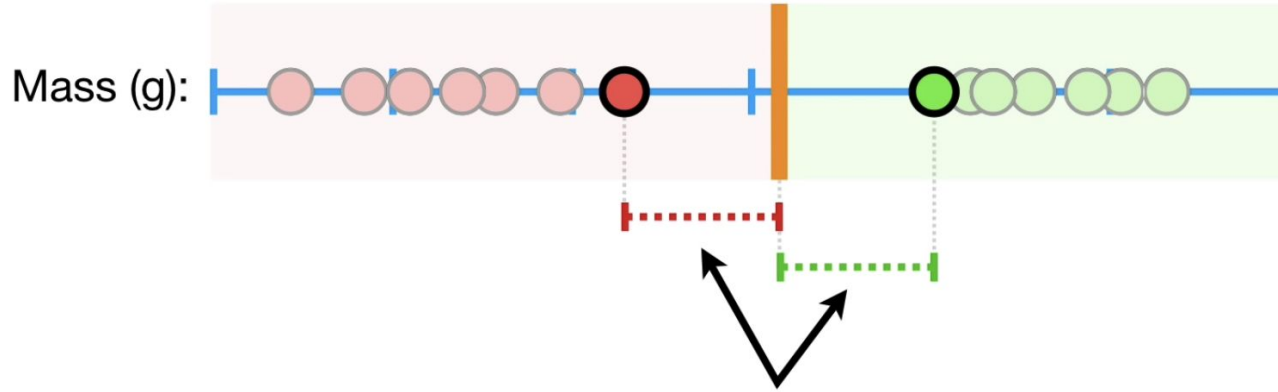


When a new observation comes in.

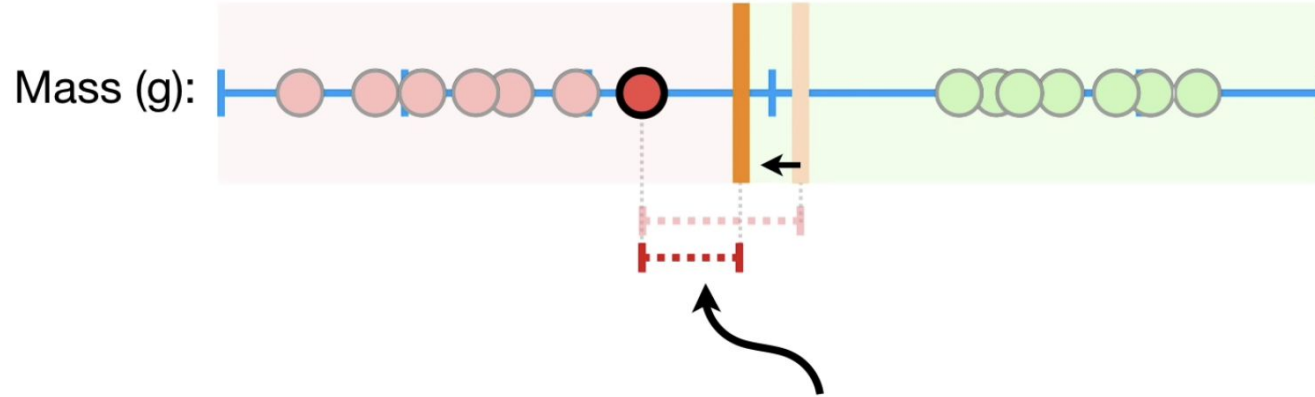


We will classify it as *not obese*.

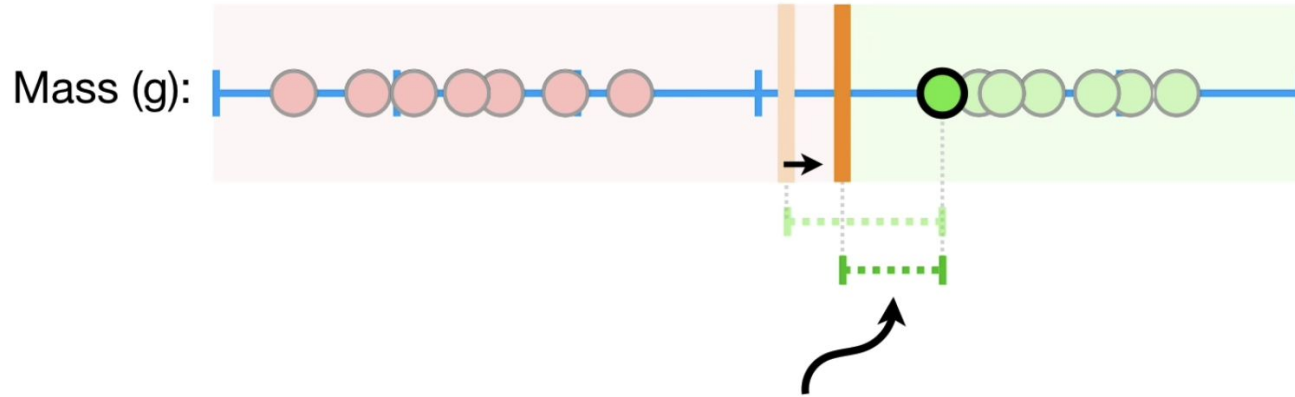
Margin



- The shortest distance between the observations and the threshold is called **margin**.
- When the threshold is halfway between the two observations, the margin is as large as it can be.

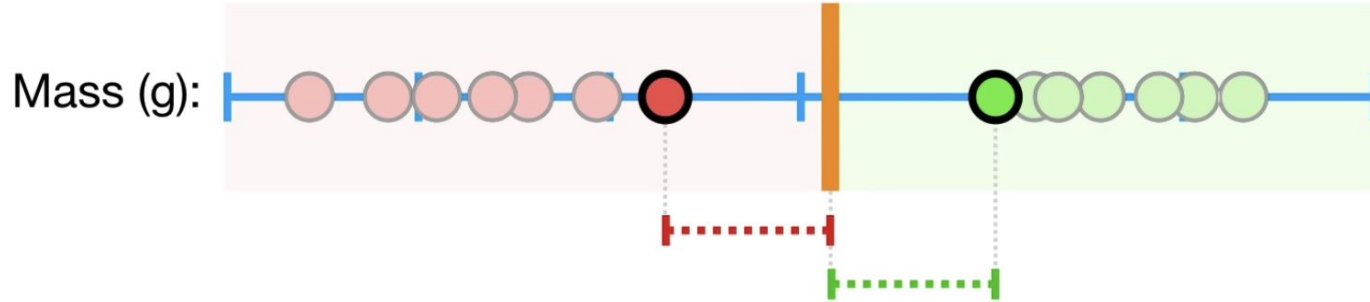


- If we move the threshold to the left a little bit, the distance between the threshold and the *not obese* observation would be smaller.



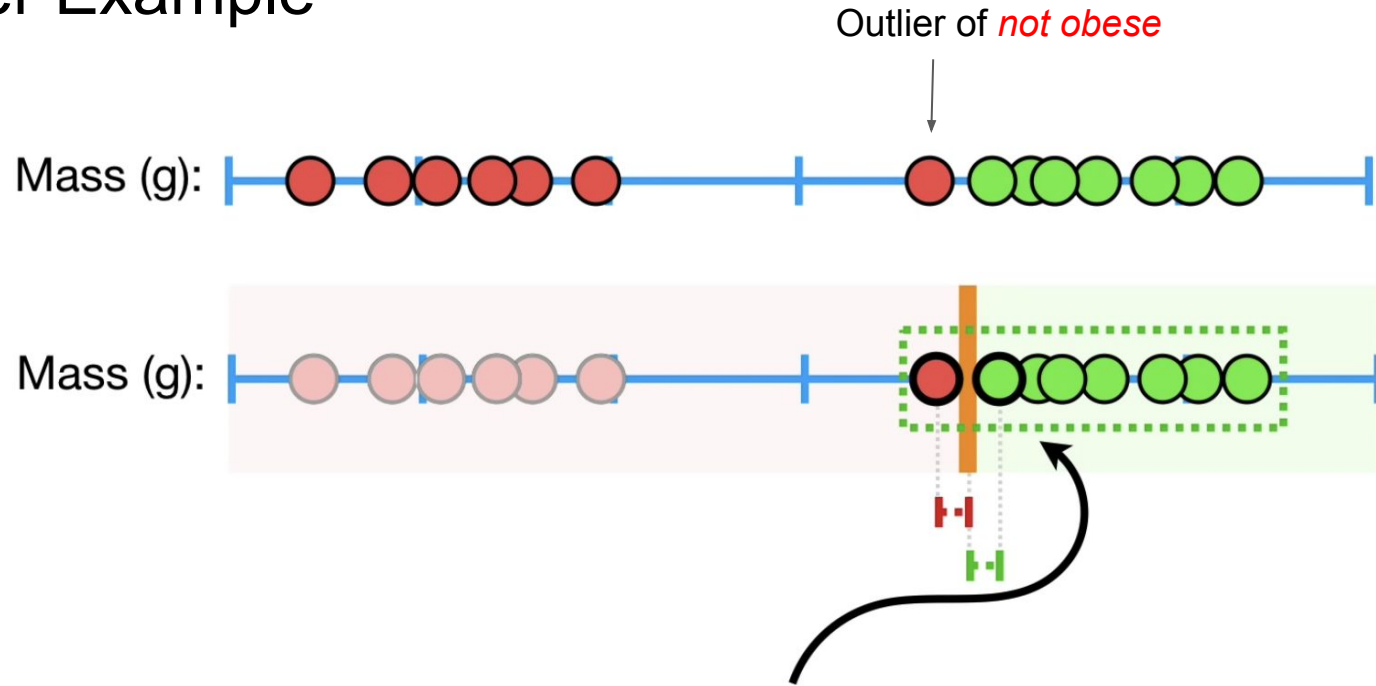
- If we move the threshold to the right a little bit, the distance between the threshold and the *obese* observation would be smaller.

Maximal Margin Classifier

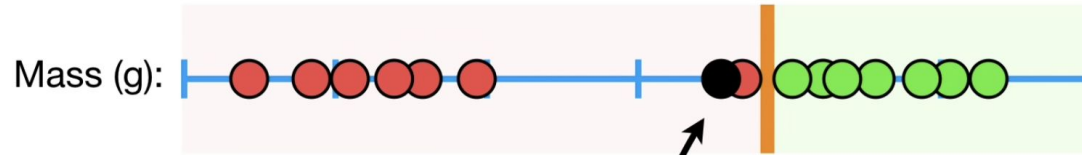


- When we use the threshold that gives us the largest margin to make classification, it's called a **Maximal Margin Classifier**

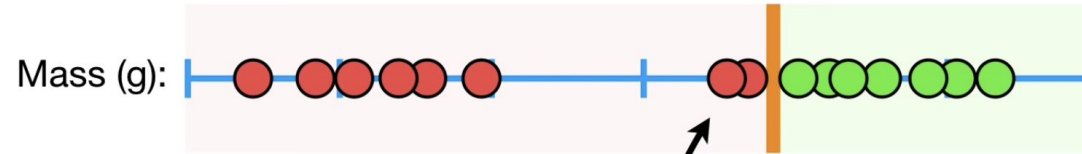
Another Example



- The **Maximum Margin Classifier** would be very close to the *obese* observations.



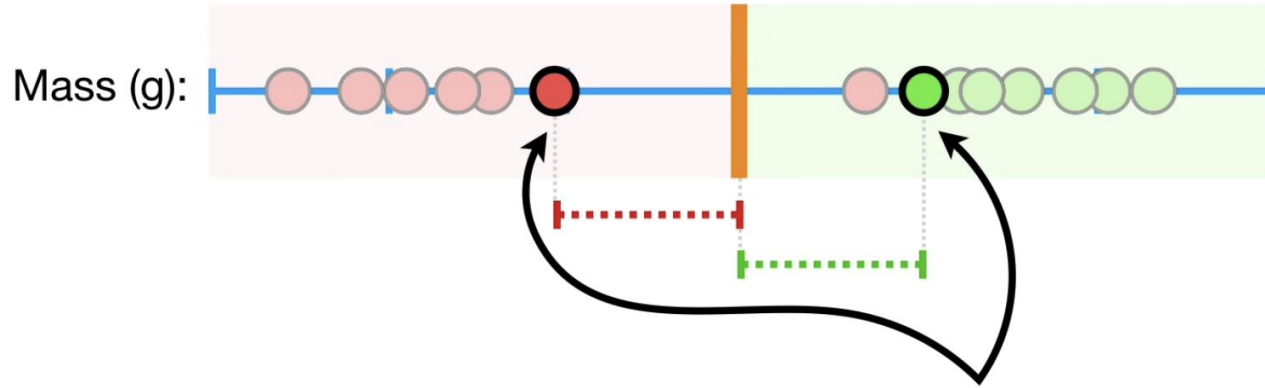
- If we have a new observation.



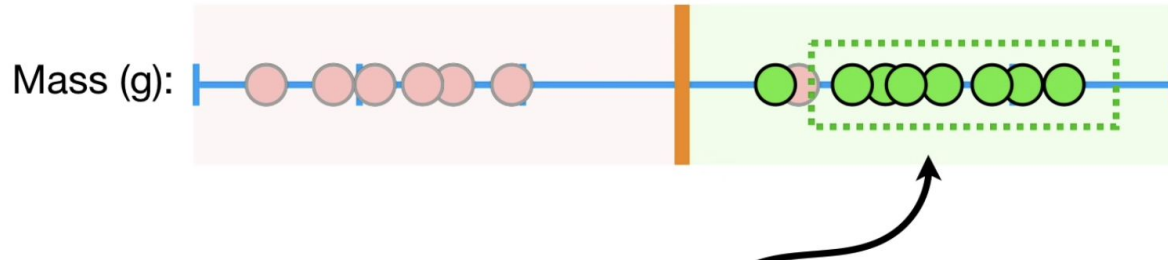
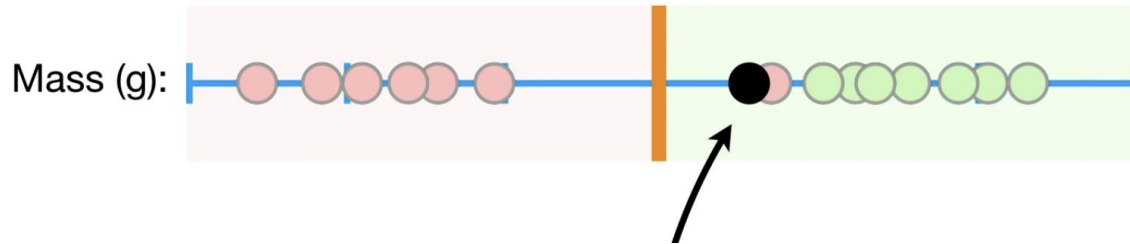
- We would classify it as *not obese* (should be *obese*).

- **Maximum Margin Classifier** is very sensitive to outliers in the training data.

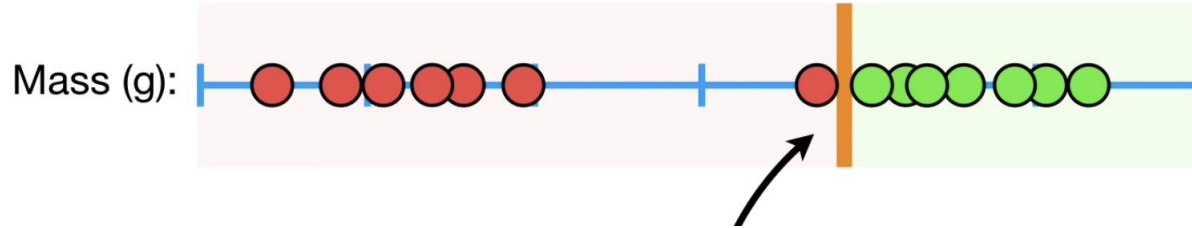
Another Better Way



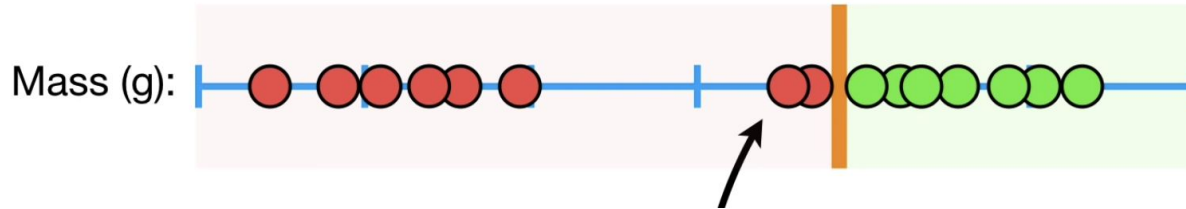
- We can improve the classifier by **allowing misclassifications** and putting the threshold halfway between these two observations.



Bias/Variance Tradeoff

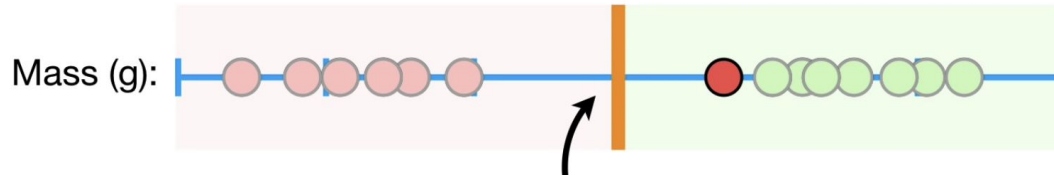


- Very sensitive to the training data (low bias).

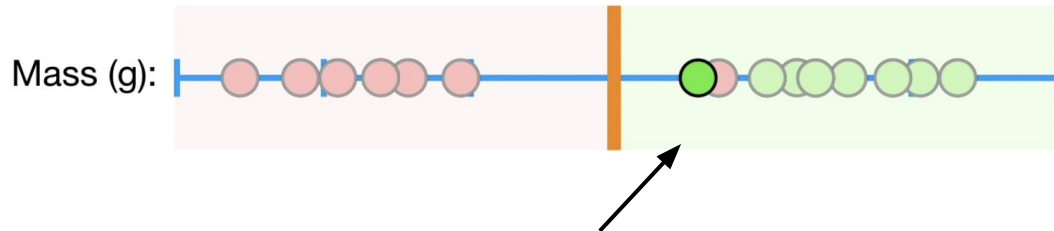


- Performed poorly on new data (high variance).

Bias/Variance Tradeoff

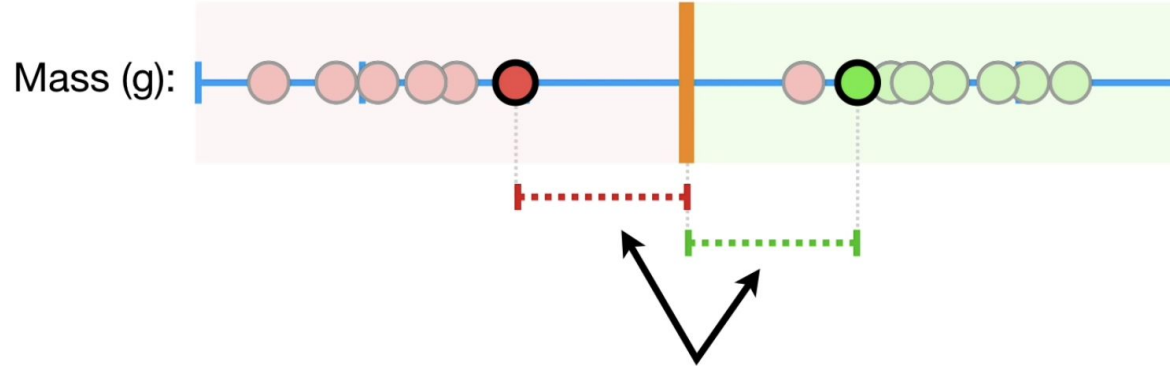


- Less sensitive to the training data and allowed misclassifications (high bias)



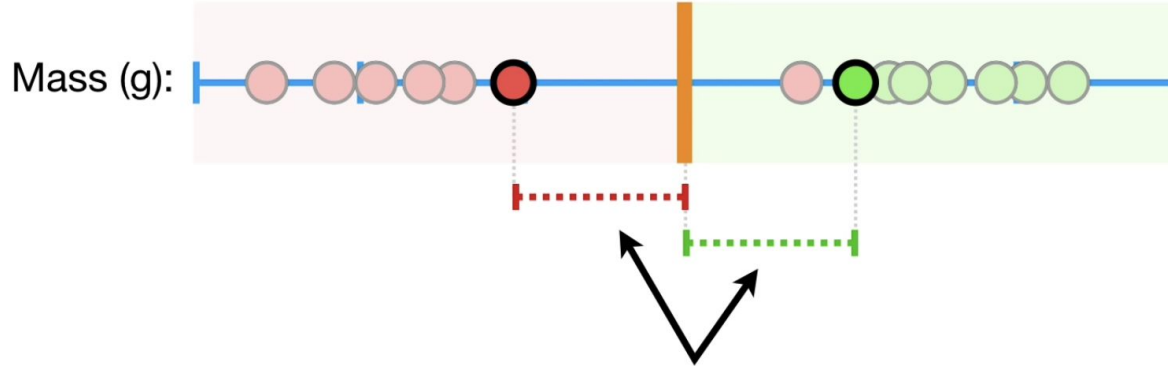
- Performed better on new data (low variance).

Soft Margin



- When we allow misclassifications, the distance between the observations and the threshold is called **Soft Margin**.
- How do we know which **Soft Margin** is better?
- **Cross Validation**

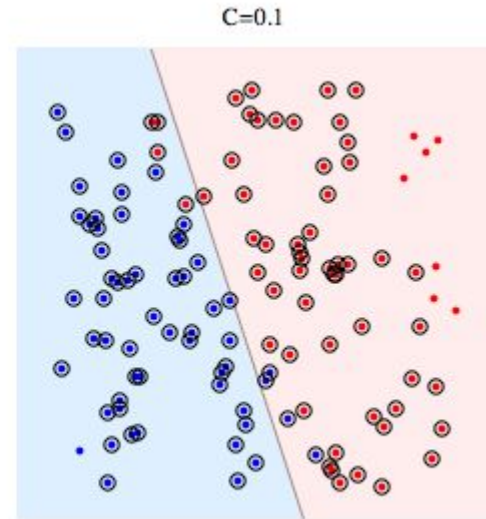
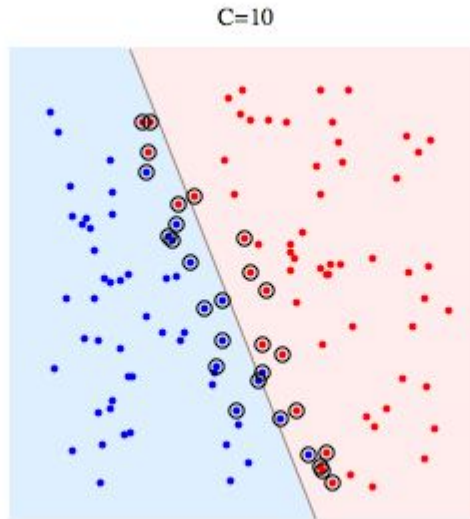
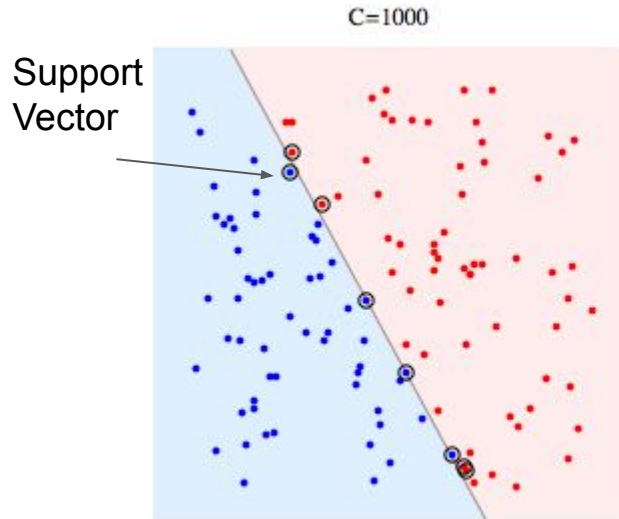
Soft Margin Classifier



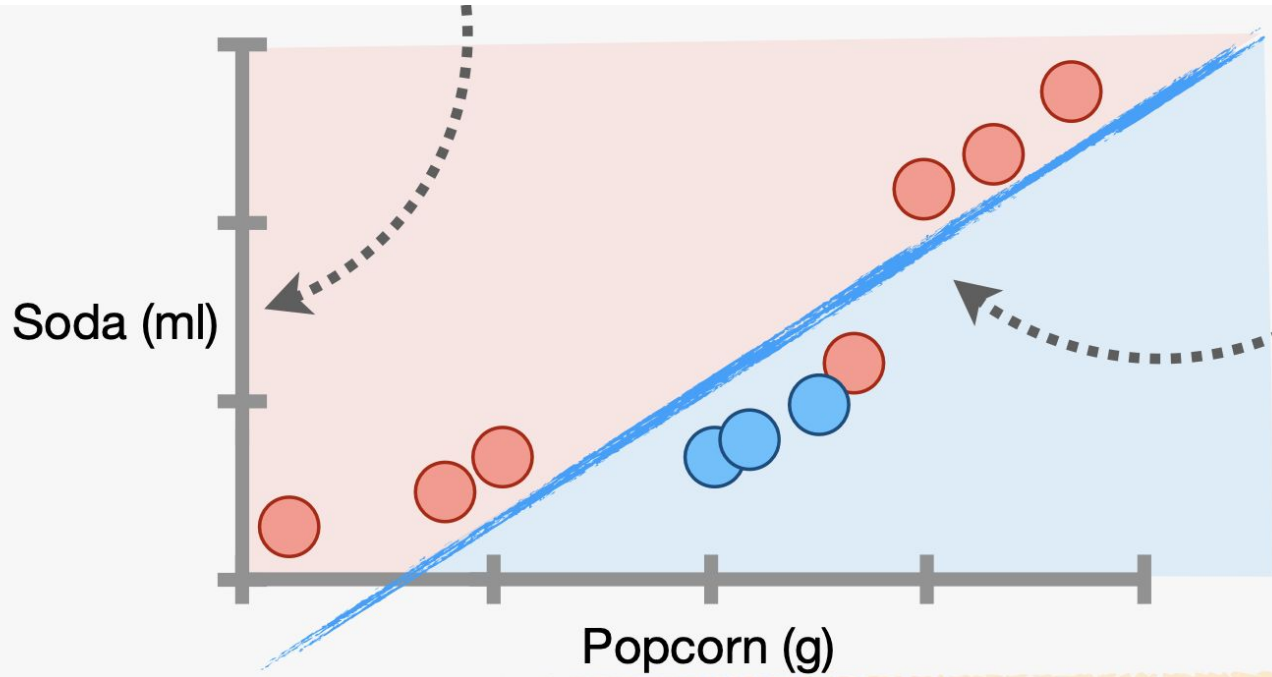
- When using **Soft Margin** to determine the threshold, we are using a **Soft Margin Classifier** aka a **Support Vector Classifier**.
- The observations on the edge or within the **Soft Margin** are called **Support Vectors**.

C Parameter (cost)

- Regularization parameter
- Larger **C** $\rightarrow$ smaller **Margin** (close to **Hard Margin**, few **Support Vectors**)
- Smaller C $\rightarrow$ larger Margin (close to **Soft Margin**, more **Support Vectors**)

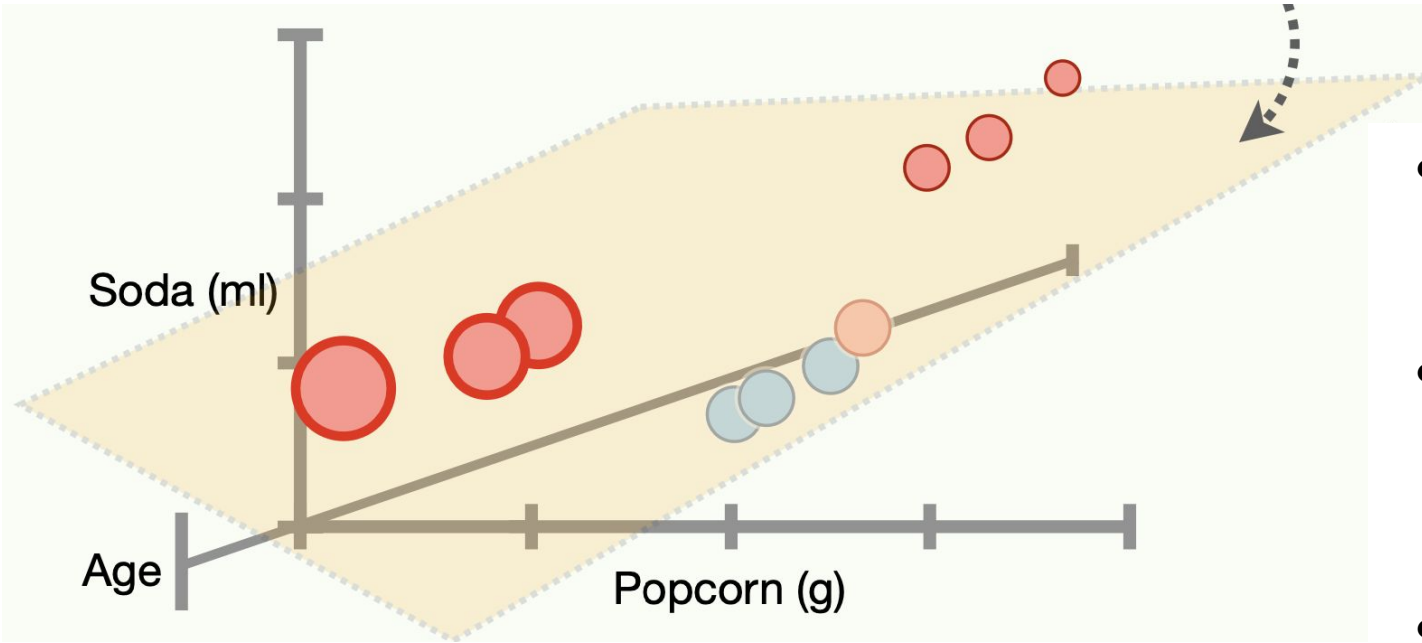


SVC on 2-Dimensional



- **SVC** would be a **straight line**.
- In other words, the **SVC** would be **1-Dimensional**.

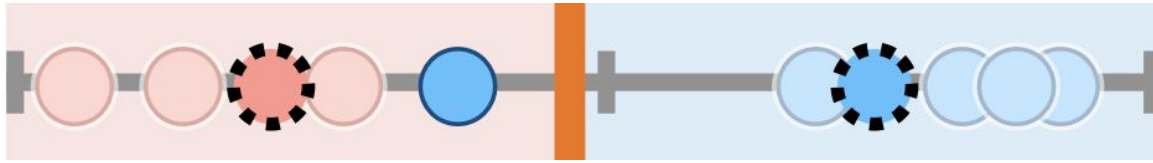
SVC on 3-Dimensional



- **SVC** would be a **2-Dimensional plane**.
- If the data is 4-Dimensional, the **SVC** would be **3-Dimensional**.
- Etc.

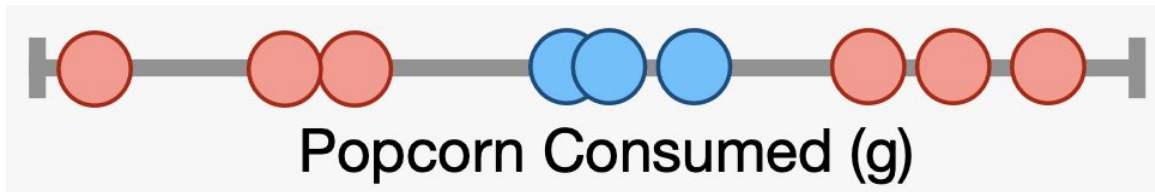
Support Vector Machine

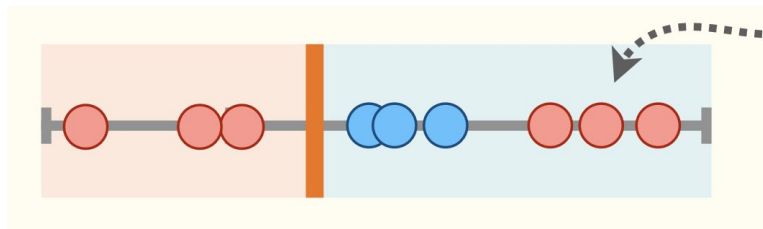
- Although SVC can handle outliers...



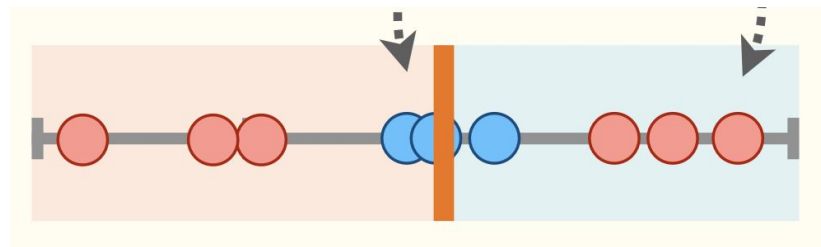
● = Loves Titanic
● = Does not love Titanic

- What if the data looks like this

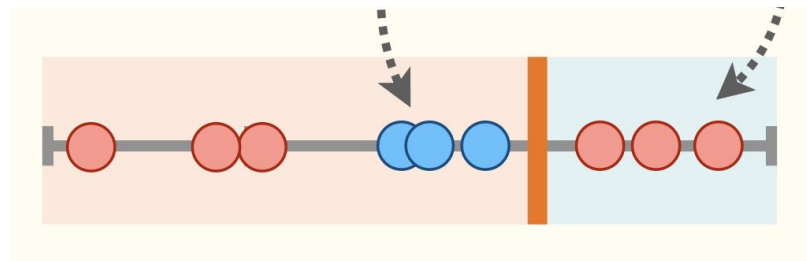




These are missclassified

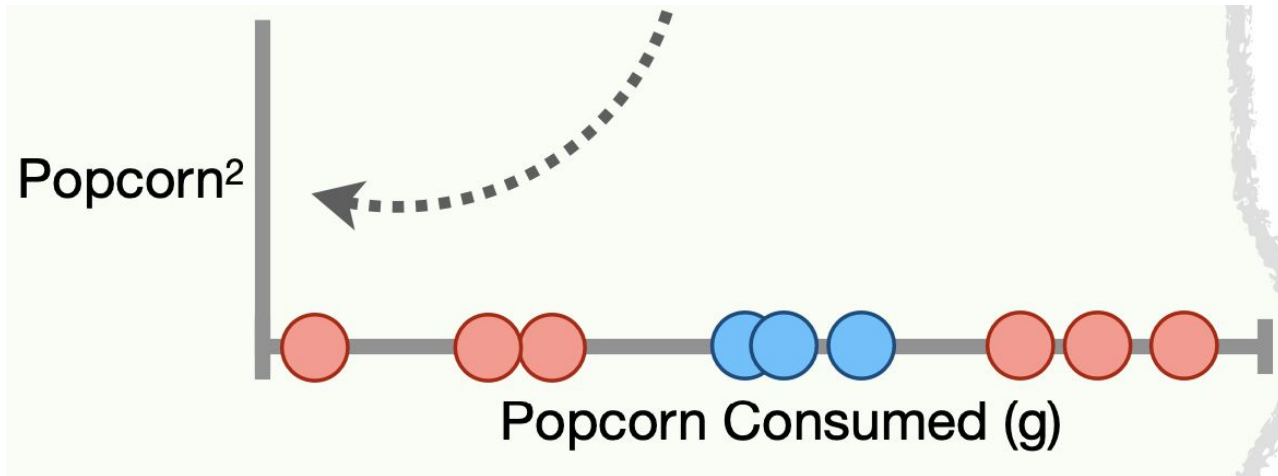


These and these are missclassified

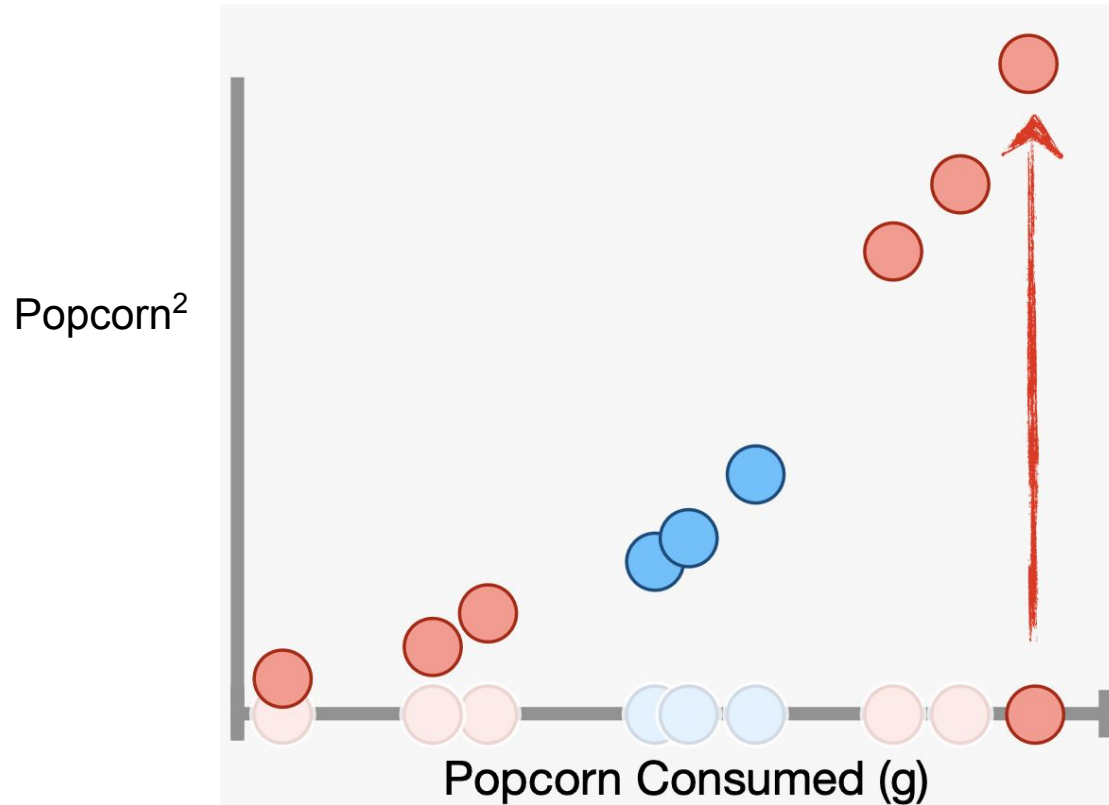


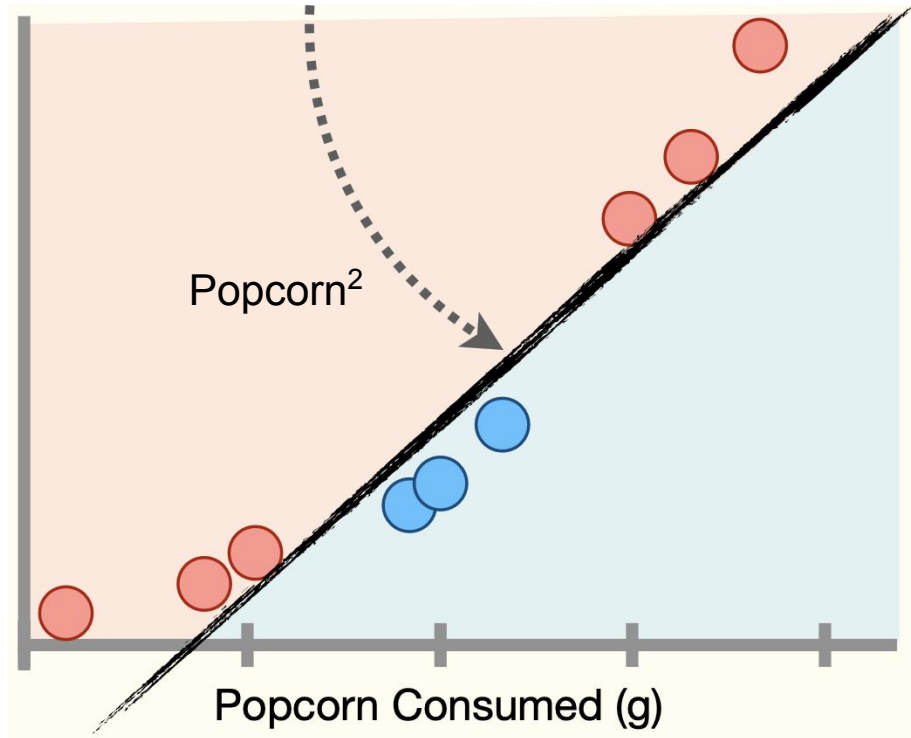
These and these are missclassified

- In polynomial regression, we create a new feature x^2



Calculate the New Features





- Now that the data are **2-Dimensional**, we can draw a **SVC** that separates the people who love Titanic from the people who do not.

The Idea Behind SVM: Polynomial Features + SVC

- Step 1: Start with low-dimensional data. In this case, we start with **1-Dimensional** data on a number line
- Step 2: Then use the existing data to create higher dimensions. In this example, we created **2- Dimensional** data by squaring the original Popcorn measurements
- Step 3: Then find a **SVC** that separates the higher dimensional data into two groups

An SVM with a polynomial kernel gets the same result without creating the new features.

Kernel Functions

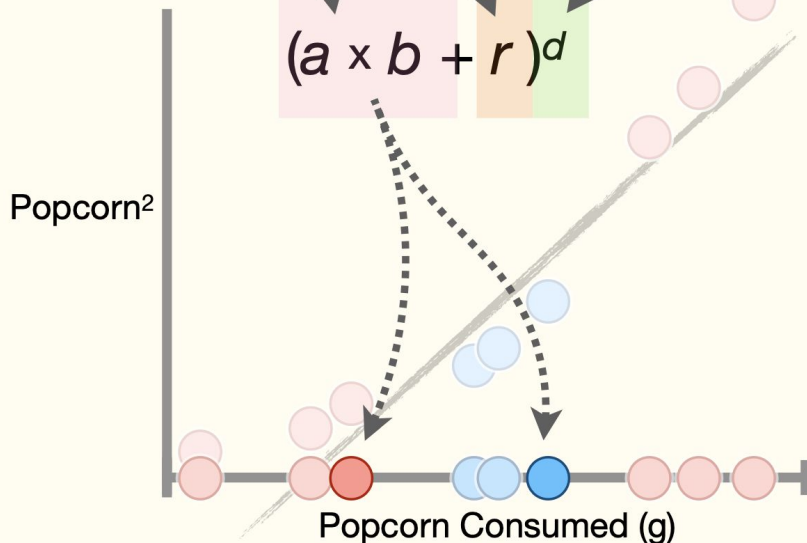
- How do we decide how to transform the data?
 - Can we use Popcorn^{1/2}?
- To make the mathematics efficient, **SVM** use something called **Kernel Functions** that we can use to systematically find **SVC** in higher dimensions
- Two of the most popular **Kernel Functions** are the **Polynomial Kernel** and the **Radial Kernel**, also known as the **Radial Basis Function**

Polynomial Kernel: $(a \times b + r)^d$

...where **a** and **b** refer to two different observations in the data...

...**r** determines the coefficient of the polynomial...

...and **d** determines the degree of the polynomial.



$$(a \times b + r)^d = (a \times b + \frac{1}{2})^2 = (a \times b + \frac{1}{2})(a \times b + \frac{1}{2})$$

$$= a^2b^2 + \frac{1}{2}ab + \frac{1}{2}ab + \frac{1}{4}$$

$$= a^2b^2 + ab + \frac{1}{4}$$

Flip the order just
for better look

$$= ab + a^2b^2 + \frac{1}{4}$$

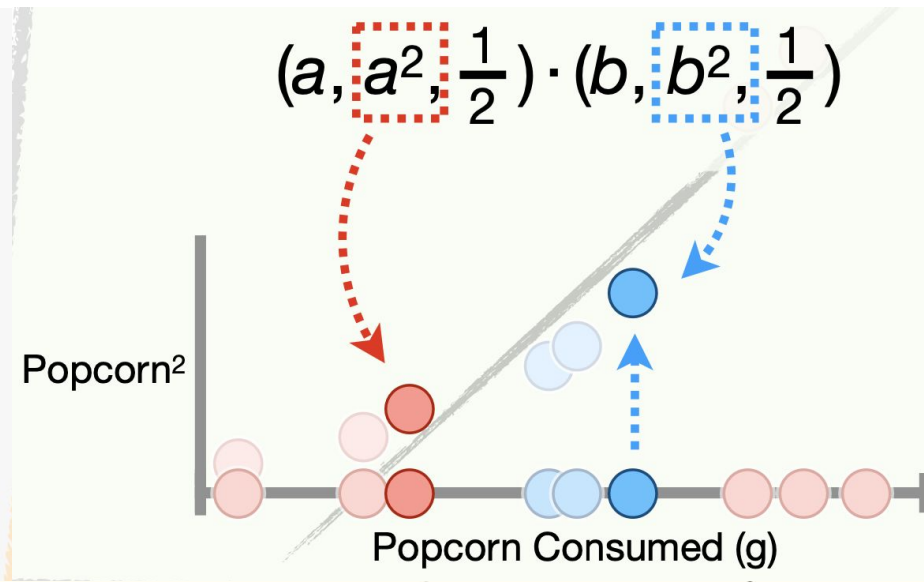
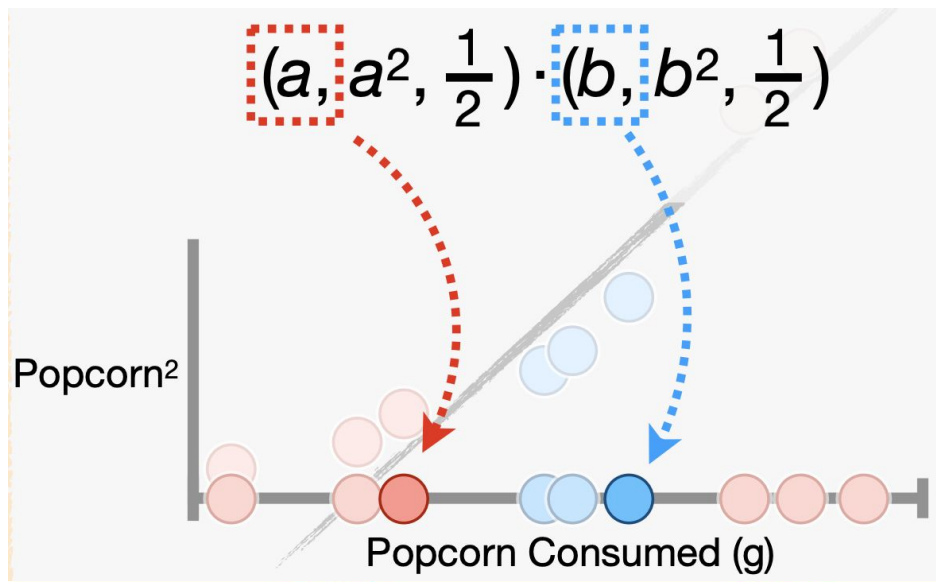
$$= (a, a^2, \frac{1}{2}) \cdot (b, b^2, \frac{1}{2})$$

Dot Product

$$\mathbf{a} = \langle a_1, a_2, a_3 \rangle \quad \mathbf{b} = \langle b_1, b_2, b_3 \rangle$$

$$\mathbf{a} \cdot \mathbf{b} = a_1b_1 + a_2b_2 + a_3b_3$$

Each observation becomes a point in
3-dimensional space



$$(a, a^2, \frac{1}{2}) \cdot (b, b^2, \frac{1}{2})$$

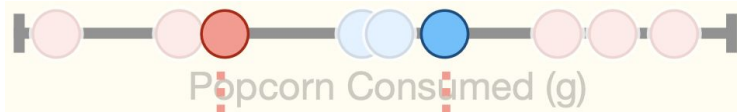
$$(a, a^2, \cancel{\frac{1}{2}}) \cdot (b, b^2, \cancel{\frac{1}{2}})$$

- The first terms are their x-axis coordinates
- The second terms are their y-axis coordinates
- The third terms are their z-axis coordinates. However, since they are the same, we can ignore them

Example

5

10

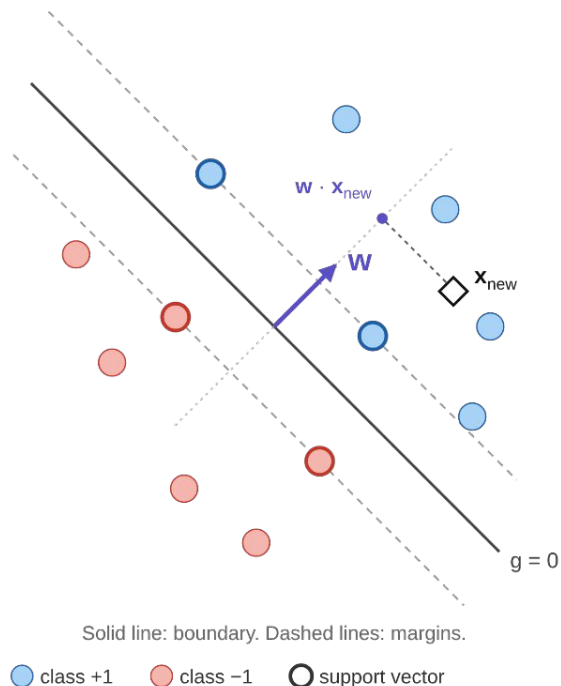


$$(a, a^2, \frac{1}{2}) \cdot (b, b^2, \frac{1}{2})$$

$$(5, 5^2, \frac{1}{2}) \cdot (10, 10^2, \frac{1}{2}) = (5 \times 10) + (5^2 \times 10^2) + (\frac{1}{2} \times \frac{1}{2}) = 2550.25$$

Kernel value

How the Boundary Decides



$$g(\mathbf{x}_{\text{new}}) = \mathbf{w} \cdot \mathbf{x}_{\text{new}} + b$$

1. $\mathbf{w}$ A direction perpendicular to the boundary.
 2. $\mathbf{w} \cdot \mathbf{x}_{\text{new}}$ How far $\mathbf{x}_{\text{new}}$ lies along $\mathbf{w}$ (its projection).
 3. b Shifts the boundary.
- If $g > 0$, predict class +1. If $g < 0$, predict class -1.

Where does $\mathbf{w}$ come from?

After training, $\mathbf{w}$ is a weighted sum of the support vectors.

$$\mathbf{w} = \sum_{i \in \text{SV}} \alpha_i y_i \mathbf{x}_i$$

- $\mathbf{x}_i$ the i -th support vector
 y_i its class label, +1 or -1
 α_i its weight, learned in training (0 for all other points)

So the +1 support vectors are added and the -1 ones are subtracted, which makes $\mathbf{w}$ point toward the +1 group.

Putting this into g gives

$$g(\mathbf{x}_{\text{new}}) = \sum_{i \in \text{SV}} \alpha_i y_i (\mathbf{x}_i \cdot \mathbf{x}_{\text{new}}) + b$$

The decision only needs dot products between $\mathbf{x}_{\text{new}}$ and the support vectors.

From Linear SVM to Kernel SVM

From the previous slide

$$g(\mathbf{x}_{\text{new}}) = \sum_{i \in \text{SV}} \alpha_i y_i (\mathbf{x}_i \cdot \mathbf{x}_{\text{new}}) + b$$



Replace each dot product with a kernel

$$g(\mathbf{x}_{\text{new}}) = \sum_{i \in \text{SV}} \alpha_i y_i K(\mathbf{x}_{\text{new}}, \mathbf{x}_i) + b$$

α_i , y_i , and b keep the same meaning as before.

Decision rule (unchanged)

If $g(\mathbf{x}_{\text{new}}) > 0$, predict class +1

If $g(\mathbf{x}_{\text{new}}) < 0$, predict class -1

The farther g is from 0, the farther the point is from the boundary.

What is $K(\mathbf{x}_{\text{new}}, \mathbf{x}_i)$?

The dot product of the two points after both are mapped to the high-dimensional space by φ .

$$K(\mathbf{x}_{\text{new}}, \mathbf{x}_i) = \varphi(\mathbf{x}_{\text{new}}) \cdot \varphi(\mathbf{x}_i)$$

The kernel gives this value directly, without computing φ .

Example: polynomial kernel

$$(a \times b + \frac{1}{2})^2 = (a, a^2, \frac{1}{2}) \cdot (b, b^2, \frac{1}{2})$$

Here $\varphi(a) = (a, a^2, \frac{1}{2})$.

For the popcorn example, $K(5, 10) = (5 \times 10 + \frac{1}{2})^2 = 2550.25$, the same value as $\varphi(5) \cdot \varphi(10)$.

A kernel SVM is a linear SVM in the high-dimensional space.

Kernel Trick

- SVM uses kernels because they remove the need to actually transform the data from low dimensions to high dimensions.
- Instead, a kernel computes the high-dimensional dot product between every pair of points directly from the original data. SVM then uses these values to find the optimal SVC.
- This technique is called **Kernel Trick**
- For the polynomial kernel, how do we choose r and d ?
 - Try different combinations and use **Cross Validation**

What Happens When $r = 1$?

$$(a \times b + 1)^2 = (a \times b + 1)(a \times b + 1)$$

$$= 2ab + a^2b^2 + 1$$

$$= (\sqrt{2} \cdot a, a^2, 1) \cdot (\sqrt{2} \cdot b, b^2, 1)$$

- Compared with $r = \frac{1}{2}$, the first coordinate is stretched by $\sqrt{2}$. The third coordinate is the same for every point, so it can be ignored
- Changing r changes the mapping, which changes where the SVC can be drawn.

Radial Kernel

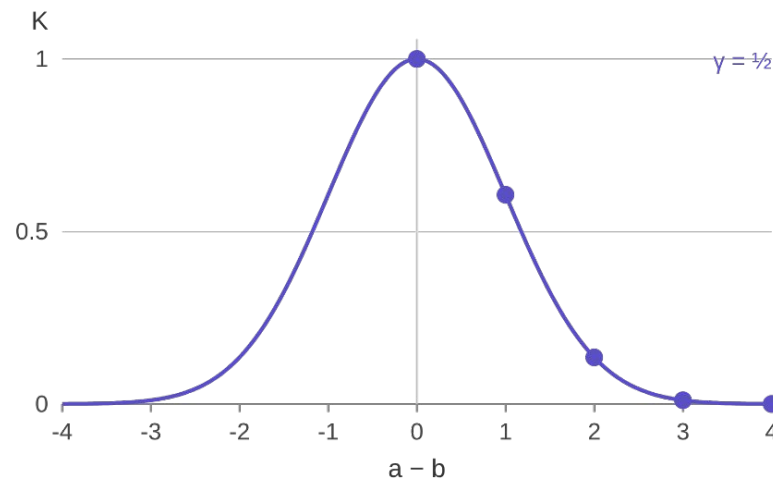
$$K(a, b) = e^{-\gamma(a-b)^2}$$

a, b two different observations in the data

$(a - b)^2$ squared distance between them

γ gamma, how fast K drops as the distance grows

Distance	$K(a, b)$ with $\gamma = \frac{1}{2}$
0	1
1	0.61
2	0.14
3	0.01
4	0.0003



Close points get K near 1. Far points get K near 0.

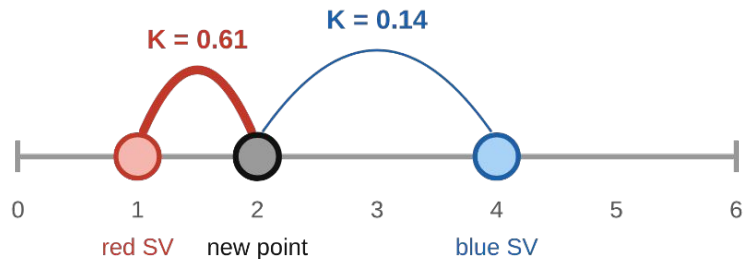
K always lies between 0 and 1, and equals 1 only when $a = b$.

The radial kernel finds an **SVC** in **infinite dimensions**, so the **Kernel Trick** is the only way to use it.

Radial Kernel Works Like K-Nearest Neighbors

Put the radial kernel into the decision function.

$$g(\mathbf{x}_{\text{new}}) = \sum_{i \in \text{SV}} \alpha_i y_i e^{-\gamma(\mathbf{x}_{\text{new}} - \mathbf{x}_i)^2} + b$$



Example ($\gamma = \frac{1}{2}$)

$$g = (1)(-1)(0.61) + (1)(+1)(0.14) + 0 = -0.47$$

$g < 0$, so the new point is classified as red.

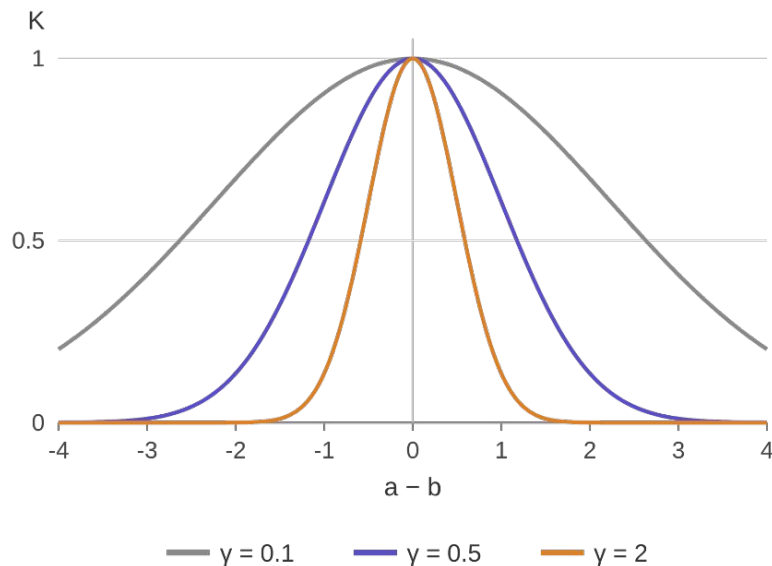
$\alpha_i = 1$ and $b = 0$ are chosen only for illustration.

Far support vectors have K close to 0, so they contribute almost nothing. The decision is driven by the **nearest** support vectors.

It works like a weighted nearest-neighbor vote.
Closer support vectors get more say.

Unlike K-nearest neighbors, only the support vectors vote.

Choosing γ



γ controls how far a point's influence reaches

Distance	$\gamma = 0.1$	$\gamma = 0.5$	$\gamma = 2$
1	0.90	0.61	0.14
2	0.67	0.14	0.0003

Large γ : only very close points matter. The boundary bends around individual points and may **overfit**.

Small γ : farther points matter too. The boundary is smoother but may **underfit**.

Find a good γ by trying many values and using **Cross Validation**.

Finding Boundry in Infinite Dimensions (Skip)

With $\gamma = \frac{1}{2}$, split the kernel into two parts.

$$e^{-\frac{1}{2}(a-b)^2} = e^{-\frac{1}{2}(a^2+b^2)} \cdot e^{ab}$$

The first part only rescales each point, so we focus on e^{ab} .

A Taylor series writes e^{ab} as an endless sum.

$$e^{ab} = 1 + ab + \frac{(ab)^2}{2!} + \frac{(ab)^3}{3!} + \dots$$

Each term is a **polynomial kernel** with $r = 0$ and degree d . It splits into one coordinate for a and one for b .

$$\frac{(ab)^d}{d!} = \frac{a^d}{\sqrt{d!}} \cdot \frac{b^d}{\sqrt{d!}}$$

Degree d	New coordinate
0	1
1	a
2	$a^2/\sqrt{2!}$
3	$a^3/\sqrt{3!}$
$\vdots$	$\vdots$

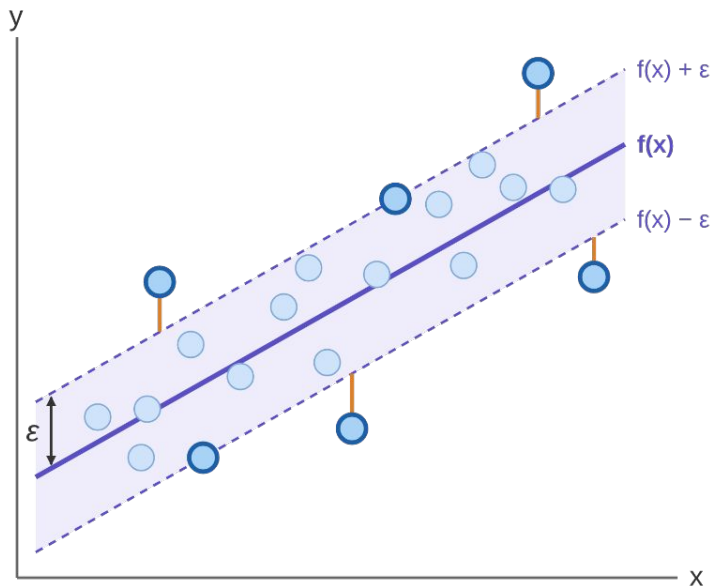
Collecting every coordinate gives

$$\varphi(a) = e^{-\frac{a^2}{2}} \left(1, a, \frac{a^2}{\sqrt{2!}}, \frac{a^3}{\sqrt{3!}}, \dots \right)$$

The radial kernel adds up polynomial kernels of **every degree**, so φ has **infinitely many** coordinates.

We can never compute φ , but the kernel gives $\varphi(a) \cdot \varphi(b)$ directly. This is why the **Kernel Trick** is essential here.

Support Vector Regression



● inside the tube ● support vector — penalty

The same ideas also work for regression.

$$f(\mathbf{x}_{\text{new}}) = \mathbf{w} \cdot \mathbf{x}_{\text{new}} + b$$

SVC finds a boundary between two groups. **SVR** fits a line that predicts a **number**.

ε-tube	A tube extending ε above and below the line.
Inside	Errors are ignored. Unlike least squares, small errors cost nothing.
Outside	Penalized by how far the point falls outside the tube.
Support vectors	Points on or outside the tube. Only they determine the line.

A larger ε gives a wider tube and a simpler model. Choose ε with **Cross Validation**.

SVR can use the same kernels to fit curves.

Advantages of SVM

- **Only the Support Vectors Matter**

- The decision depends only on the support vectors. Points far from the boundary do not affect it, and the final model stays **compact**

- **Handles Non-Linear Patterns with Kernels**

- Kernels let SVM draw curved boundaries without computing new coordinates
- It can work in very high or even infinite dimensions at a **manageable cost**

- **Works Well with Many Features and Few Samples**

- Examples include text (one feature per word) and gene expression data
- Many lines can separate the training data perfectly, and it is easy to pick one that fits too tightly
- SVM always picks the line with the widest margin, which tends to hold up better on new data

Disadvantages of SVM

- **Slow to Train on Large Datasets**

- During training, SVM does not yet know which points will be support vectors, so it must compare every pair of training points
- Doubling the data roughly quadruples the number of pairs, so training on hundreds of thousands of samples becomes impractical

- **Requires Careful Parameter Tuning**

- Results depend on the kernel and its parameters (r and d , γ , ϵ)

- **Hard to Interpret with Non-Linear Kernels**

- With a linear kernel, each feature has one weight in w , so its effect is easy to see.
- With polynomial or radial kernels, the decision happens in a high-dimensional space and has no single weight per original feature.
- This makes it hard to explain which features drove a decision, which matters in fields such as medicine or finance

- **No Direct Probability Output**

- SVM gives a score $g(x)$ and uses its sign to decide. It does not say how likely each class is